iPay Payment Gateway

Integration Document

Version 1.0

Table of Contents

1. Introduction

The usage of E-Commerce has increased exponentially in the recent past. The iPay Payment Gateway solution looks to empower the growth of all businesses with a website by providing a payment processing platform to ride the E-Commerce wave. The iPay Payment Gateway can be integrated to your website easily with minimum technical effort by following the simple instructions in this document. The successful integration to the iPay Payment Gateway will allow businesses to accept payments from their customers via Payment Cards, via iPay and other LankaQR compliant apps in the country.

This document will provide an easy to follow, step-by-step guide to the technical integration work required to enable this feature to the merchant e-commerce website in the iPay Sandbox environment. Upon successful integration in the Sandbox environment, the business will be required to contact the iPay Merchant Management Team to complete the iPay merchant registration and enable this product in the live environment. The steps followed in the Sandbox environment will simply have to be replicated in the live environment after successful registration.

2. How iPay Payment Gateway Works

iPay provides HTML form-based POST API to integrate iPay with your website. You can initiate a payment request by submitting required information to that API from your website and it will redirect your customer to the iPay Payment Gateway interface.

Customers can initiate payments by selecting any payment source offered by the merchant & providing the required details for the selected payment source.

1. Payment Card – Enter & submit required card details such as Name, Card Number, CVV, and Expiry Date to verify the details.
2. Lanka QR - Scan the LankaQR code and make the payment by any LankaQR compliant mobile application.
3. iPay - Enter user mobile number & email address as account information & then the user will receive a push notification to iPay in order to authenticate the payment.

Once the payment is processed, iPay notifies your call-back API about the payment status as a server callback. Here, iPay generates a checksum and sends it with response parameters to ensure integrity and you can use those response parameters & checksum to verify and update your system accordingly.

3. Prerequisites

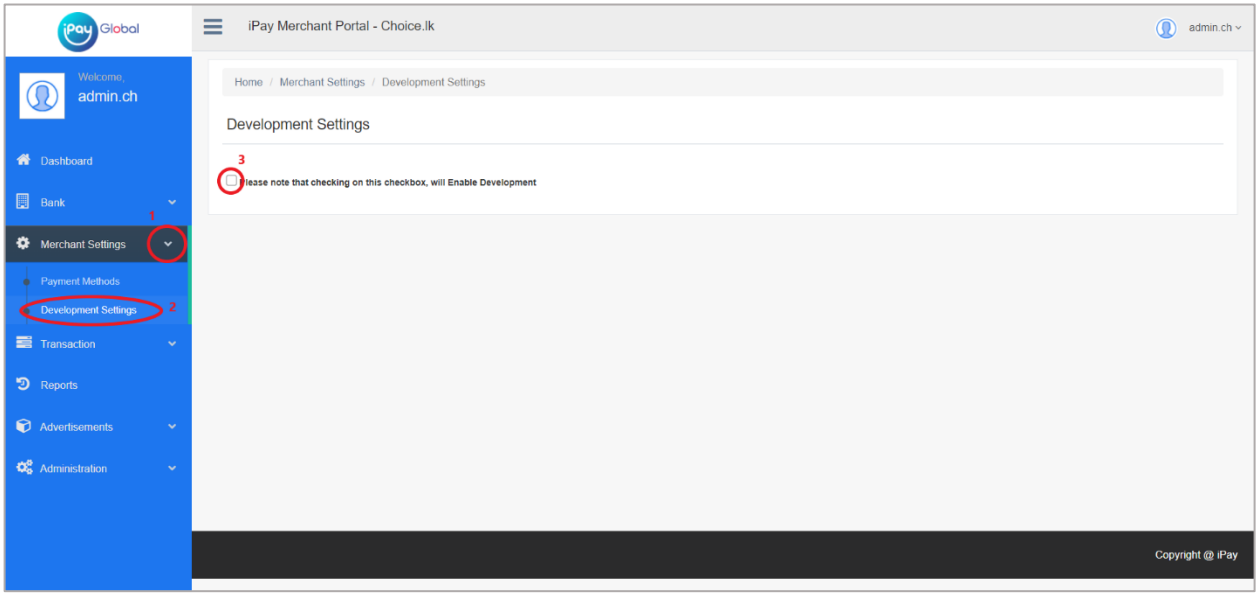
1. You as the web merchant should have your own custom e-commerce web application as you need code level integration.
2. You as the web merchant should register as a Merchant in the iPay Sandbox by providing the required information to obtain the Web payment token.
3. You should have the iPay sandbox mobile application or any other Lanka QR compliant payment application connected with the Lanka Pay sandbox. (You can download the iPay sandbox mobile application from the Admin Portal after registration)

4. Getting Started

1. Go to the iPay Sandbox website using the following URL.
   Sandbox: https://sandbox.ipay.lk/ipayMerchantApp/enroll/businessType

2. Register as a merchant and log in to the Sandbox Merchant admin portal by using the username and password approved at registration.

3. Go to the "**Merchant Settings"** option on the user side menu and click on **"Development Settings"**.
   Then the page will display the screen shown below. Enable Development Settings by clicking on the checkbox shown below.

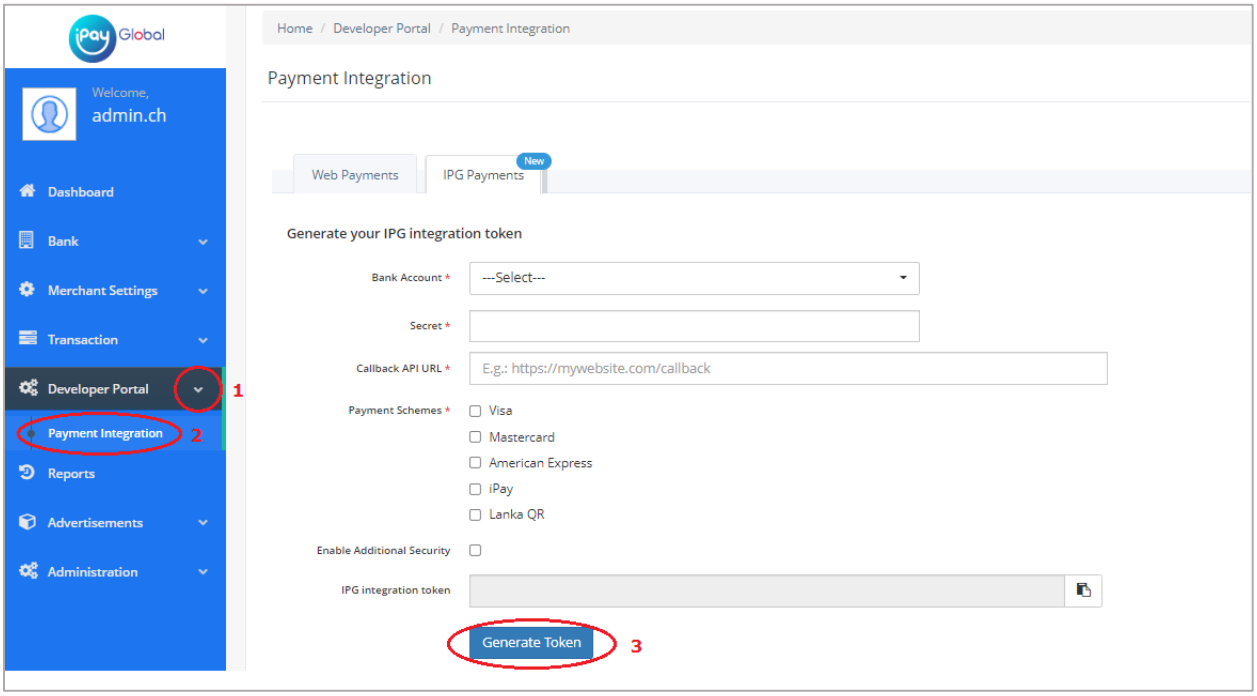


4. After enabling the merchant for development, the **"Developer Portal"** will automatically display on the user side menu as shown below. Then select the **"Payment Integration" "IPG Payments"** option and generate an IPG integration token by providing the required details.

   Then Merchant should provide value for the **"Secret"** option and this secret is used to generate the **Checksum**.

   Merchant should provide a "**Call back API URL**" and this will be used to notify the Payment Status.

   Note: This IP address or domain based **HTTPS** URL must be **publicly accessible.**



5. Select the payment schemes you want to enable for your website.

6. Enable Additional Security (Optional) – Merchant can enable additional security for the checkout HTML form to ensure the integrity of Order ID and Transaction Amount. Here, the Merchant needs to generate a checksum using the below given combination of parameters and send it to the checkout HTML form.

   Message = IPG Integration Token + Order ID + Transaction Amount

   NOTE: Please refer to Section 5.3 for additional information on checksum generation

7. The IPG Integration Token will be automatically generated when clicked on the **"Generate Token"** button which will be specific for the merchant. The merchant should use this **IPG Integration Token** when setting up iPay Payment checkout API.

5. Setting up Development Environment

5.1 Add Checkout HTML form to your website

iPay provides HTML form-based POST API to integrate iPay with your website. Here you need to add an HTML form to your website and submit the required parameters to iPay Payment Gateway interface. After submitting the form, the customer will be redirected to the iPay Payment Gateway interface securely.

The HTML form information is as follows.

Action URLs

Environment	URL
Sandbox	https://sandbox.ipay.lk/ipg/checkout
Live	https://ipay.lk/ipg/checkout

From Parameters (The data type of all parameters are alphanumeric)

Parameter Name	Description	Max Length	Mandatory?
merchantWebToken	The IPG Integration token generated in the Merchant Admin Portal	N/A	YES
totalAmount	Total payment amount	10	YES
subMerchantReference	Applicable for Aggregator Merchants only	8	NO
orderId	Unique Order ID generated by merchant	100	YES
orderDescription	Description of the Order	200	NO
returnUrl	URL to redirect users when proceed the payment	N/A	YES
cancelUrl	URL to redirect users when cancel the payment or allowed time period has passed	N/A	YES
customerName	Customer's Name	250	NO
customerPhone	Customer's Phone No	15	NO
customerEmail	Customer's Email	100	NO
merchantParam1	Additional parameter 1 set by Merchant	200	NO
merchantParam2	Additional parameter 2 set by Merchant	200	NO
paymentMethod	If only 1 payment scheme to be displayed	5	NO
checksum	Checksum for additional security	N/A	NO

* The data type of all parameters are alphanumeric

Payment Scheme Filtrations

Merchant can filter any payment scheme for a specific checkout request if required by passing payment method to the checkout HTML form. In the Merchant web page, if you require only a particular payment scheme to be displayed, then you will be required to pass the respective Code below to the checkout form as the “paymentMethod” parameter. Then other payment schemes will not be visible to the customer for selection.

Code	Payment Scheme
VISA	Visa
MC	Mastercard
IPAY	iPay
LQR	Lanka QR

Code Sample

```
<html>
<body>
<form method="POST" action="https://sandbox.ipay.lk/ipg/checkout">
   <input type="hidden" name="merchantWebToken" value="eyJhbGciOiJIUz..."> <!-- Replace your web token -->
   <input type="hidden" name="orderId" value="OID123456">
   <input type="hidden" name="orderDescription" value="My Order"> <!-- Optional -->
   <input type="hidden" name="returnUrl" value="http://mywebsite.com/return?orderId=OID123456">
   <input type="hidden" name="cancelUrl" value="http://mywebsite.com/cancel?orderId=OID123456">
   <input type="hidden" name="subMerchantReference" value=""> <!-- Optional -->
   <table>
      <tr>
         <td>Total Amount</td>
         <td>:</td>
         <td><input type="text" name="totalAmount" value="750"></td>
      </tr>
      <tr>
         <td>Customer Name </td>
         <td>:</td>
         <td><input type="text" name="customerName" value="Ravindu Fernando"></td>
      </tr>
      <tr>
         <td>Customer Mobile</td>
         <td>:</td>
         <td><input type="text" name="customerPhone" value="0701234567"></td>
      </tr>
      <tr>
         <td>Customer Email</td>
         <td>:</td>
         <td><input type="text" name="customerEmail" value="myemail@mail.com"></td>
      </tr>
   </table>
   <br>
   <input type="submit" value="Checkout Now">
</form>
</body>
</html>
```

5.2 Capture the Transaction

Once the payment is authorized by the customer, iPay will notify the payment status to your call-back API. Payment notification will contain the following data as **JSON** POST parameters, so you need to make sure the call-back API URL you set is accepting these parameters on a POST request. After capturing the payment status you need to verify the payment status and update your database accordingly. **Here you need to cross check your order amount matches with the transaction amount sent by call-back API to make sure the customer has paid the correct amount.**

Note:

iPay redirects the customer back to your website **returnUrl** which you provided, only after we receive the **Success** response from your **Call-back API**.

We do not send any parameters with this return URL and you must include the required query parameters to your return URL such as Order ID, in order to obtain payment status from your database when the customer redirects back to the return URL.

Call-back API Parameters

Parameter Name	Description	Max Length
transactionReference	Payment reference number for particular transaction generated by iPay	20
orderId	Order ID sent by the Merchant to iPay Payment checkout interface	100
transactionAmount	Total transaction amount sent by the Merchant to proceed the transaction	12
creditedAmount	Amount credited to Merchant's bank account after deducting commissions	12
transactionStatus	Status of the transaction (A, P, D)	2
transactionMessage	Message for the transaction	200
transactionTimeInMillis	Transaction initiated time in milliseconds	20
merchantParam1	Additional parameter 1 provided by the Merchant at checkout	200
merchantParam2	Additional parameter 2 provided by the Merchant at checkout	200
checksum	Checksum generated using payload data	200

* The data type of all parameters should be alphanumeric

Transaction Status Codes

A - Accepted (Transaction process is successfully completed)
P - Pending (Money successfully deducted from the customer bank account, but unable to transfer to merchant's account. The amount will be transferred at the End of Day)
D - Decline (Transaction failed)

5.3 Verifying Payment Status

It is important to verify the Payment notification before taking any action on the payment response. You can do the verification using the checksum parameter generated and sent by iPay.

The Checksum is generated using the HMAC SHA256 algorithm, using the below given combination of parameters.

message = transactionReference + orderId + transactionTimeInMillis + transactionAmount + transactionStatus

checksum = H (key1 || H (key2 || message))

Note: Please refer the ***Appendix*** section for more information on creating base64 hashes using HMAC SHA256 in different languages.

Once you receive the payment notification from iPay, you can locally generate this checksum using above mentioned parameters and the secret you have locally. Your locally generated checksum should be equal to the checksum sent by iPay if the payment notification is valid.

In addition once we invoke the call-back API,
You should return HTTP response code 200 (OK) if your operation is successful from your end.

Any other HTTP response code will be taken as a failure response.

6. Card Payments Test Details

Please use the below test data to test Debit/Credit card transactions on Sandbox environment.

Test Cards	Card Number	3-D Secure Enrolled
Mastercard	5123450000000008	Y
	2223000000000007	Y
	5111111111111118	N
	2223000000000023	N
Visa	4508750015741019	Y
	4012000033330026	N

Expiry Date	Transaction Response Gateway Code
01 / 39	APPROVED
05 / 22	DECLINED
04 / 27	EXPIRED_CARD
08 / 28	TIMED_OUT
01 / 37	ACQUIRER_SYSTEM_ERROR
02 / 37	UNSPECIFIED_FAILURE
05 / 37	UNKNOWN

CSC/CVV	CSC/CVV Response Gateway Code
100	MATCH
101	NOT_PROCESSED
102	NO_MATCH

7. Live Implementation

Upon successful integration to the Sandbox environment and if the business entity wishes to deploy this integration to the live environment, the merchant is 1st required to register as an iPay merchant. For this purpose, you are required to contact your preferred LOLC Branch or contact the following member from the iPay Merchant Management Team.

Mr. Tuan Dhamin
Mobile: 077 659 5622
Email: DhaminS@lolcfinance.com

After the iPay merchant registration is successfully completed, you are required to follow these same steps in your live environment.

Appendix

Javascript HMAC SHA256

Dependent upon an open source js library called http://code.google.com/p/crypto-js/.

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/crypto-js/3.1.9-1/crypto-js.min.js
"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/crypto-js/3.1.9-1/hmac-sha256.min.
js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/crypto-js/3.1.9-1/enc-base64.min.j
s"></script>

<script>
  var hash = CryptoJS.HmacSHA256("Message", "secret");
  var hashInBase64 = CryptoJS.enc.Base64.stringify(hash);
  document.write(hashInBase64);
</script>
```

PHP HMAC SHA256

PHP has built in methods for hash_hmac (PHP 5) and base64_encode (PHP 4, PHP 5) resulting in no outside dependencies. Say what you want about PHP but they have the cleanest code for this example.

```
$s = hash_hmac('sha256', 'Message', 'secret', true);
echo base64_encode($s);
```

Java HMAC SHA256

Dependent on Apache Commons Codec to encode in base64.

```
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import org.apache.commons.codec.binary.Base64;

public class ApiSecurityExample {
  public static void main(String[] args) {
    try {
     String secret = "secret";
     String message = "Message";

     Mac sha256_HMAC = Mac.getInstance("HmacSHA256");
     SecretKeySpec secret_key = new SecretKeySpec(secret.getBytes(), "HmacSHA256");
     sha256_HMAC.init(secret_key);

     String hash = Base64.encodeBase64String(sha256_HMAC.doFinal(message.getBytes()));
     System.out.println(hash);
    }
    catch (Exception e){
     System.out.println("Error");
    }
   }
}
```

Groovy HMAC SHA256

It is mostly java code but there are some slight differences. Adapted from Dev Takeout - Groovy HMAC/SHA256 representation.

```
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.security.InvalidKeyException;

def hmac_sha256(String secretKey, String data) {
 try {
    Mac mac = Mac.getInstance("HmacSHA256")
    SecretKeySpec secretKeySpec = new SecretKeySpec(secretKey.getBytes(), "HmacSHA256"
)
    mac.init(secretKeySpec)
    byte[] digest = mac.doFinal(data.getBytes())
    return digest
   } catch (InvalidKeyException e) {
    throw new RuntimeException("Invalid key exception while converting to HMac SHA256"
)
  }
}

def hash = hmac_sha256("secret", "Message")
encodedData = hash.encodeBase64().toString()
log.info(encodedData)
```

C# HMAC SHA256

```
using System.Security.Cryptography;

namespace Test
{
  public class MyHmac
  {
    private string CreateToken(string message, string secret)
    {
      secret = secret ?? "";
      var encoding = new System.Text.ASCIIEncoding();
      byte[] keyByte = encoding.GetBytes(secret);
      byte[] messageBytes = encoding.GetBytes(message);
      using (var hmacsha256 = new HMACSHA256(keyByte))
      {
        byte[] hashmessage = hmacsha256.ComputeHash(messageBytes);
        return Convert.ToBase64String(hashmessage);
      }
    }
  }
}
```

Objective C and Cocoa HMAC SHA256

Most of the code required was for converting to bae64 and working the NSString and NSData data types.

```
#import "AppDelegate.h"
#import <CommonCrypto/CommonHMAC.h>

@implementation AppDelegate

- (void)applicationDidFinishLaunching:(NSNotification *)aNotification {
 NSString* key = @"secret";
 NSString* data = @"Message";

 const char *cKey = [key cStringUsingEncoding:NSASCIIStringEncoding];
 const char *cData = [data cStringUsingEncoding:NSASCIIStringEncoding];
 unsigned char cHMAC[CC_SHA256_DIGEST_LENGTH];
 CCHmac(kCCHmacAlgSHA256, cKey, strlen(cKey), cData, strlen(cData), cHMAC);
 NSData *hash = [[NSData alloc] initWithBytes:cHMAC length:sizeof(cHMAC)];

 NSLog(@"%@", hash);

 NSString* s = [AppDelegate base64forData:hash];
 NSLog(s);
}

+ (NSString*)base64forData:(NSData*)theData {
 const uint8_t* input = (const uint8_t*)[theData bytes];
 NSInteger length = [theData length];

 static char table[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789
+/=";

 NSMutableData* data = [NSMutableData dataWithLength:((length + 2) / 3) * 4];
 uint8_t* output = (uint8_t*)data.mutableBytes;

 NSInteger i;
 for (i=0; i < length; i += 3) {
 NSInteger value = 0;
 NSInteger j;
 for (j = i; j < (i + 3); j++) {
 value <<= 8;

 if (j < length) {  value |= (0xFF & input[j]);  }  }  NSInteger theIndex = (i / 3) *
4;  output[theIndex + 0] = table[(value >> 18) & 0x3F];
 output[theIndex + 1] = table[(value >> 12) & 0x3F];
 output[theIndex + 2] = (i + 1) < length ? table[(value >> 6) & 0x3F] : '=';
 output[theIndex + 3] = (i + 2) < length ? table[(value >> 0) & 0x3F] : '=';
 }

 return [[NSString alloc] initWithData:data encoding:NSASCIIStringEncoding]; }

@end
```

Go programming language - Golang HMAC SHA256

Try it online in your browser with Play GoLang crypto/hmac package

```
package main

import (
    "crypto/hmac"
    "crypto/sha256"
    "encoding/base64"
    "fmt"
)

func ComputeHmac256(message string, secret string) string {
    key := []byte(secret)
    h := hmac.New(sha256.New, key)
    h.Write([]byte(message))
    return base64.StdEncoding.EncodeToString(h.Sum(nil))
}

func main() {
    fmt.Println(ComputeHmac256("Message", "secret"))
}
```

Ruby HMAC SHA256

Requires openssl and base64.

```
require 'openssl'
require "base64"

hash  = OpenSSL::HMAC.digest('sha256', "secret", "Message")
puts Base64.encode64(hash)
```

Python (2.7) HMAC SHA256

```
import hashlib
import hmac
import base64

message = bytes("Message").encode('utf-8')
secret = bytes("secret").encode('utf-8')

signature = base64.b64encode(hmac.new(secret, message, digestmod=hashlib.sha256).diges
t())
print(signature)
```

Python (3.7) HMAC SHA256

```
import hashlib
import hmac
import base64

message = bytes('Message', 'utf-8')
secret = bytes('secret', 'utf-8')

signature = base64.b64encode(hmac.new(secret, message, digestmod=hashlib.sha256).diges
t())
print(signature)
```

Perl HMAC SHA256

By convention, the Digest modules do not pad their Base64 output. To fix this you can test the length of the hash and append equal signs "=" until it is the length is a multiple of 4. We will use a modulus function below.

```
use Digest::SHA qw(hmac_sha256_base64);
$digest = hmac_sha256_base64("Message", "secret");

# digest is currently: qnR8UCqJggD55PohusaBNviGoOJ67HC6Btry4qXLVZc

# Fix padding of Base64 digests
while (length($digest) % 4) {
    $digest .= '=';
}

print $digest;
# digest is now: qnR8UCqJggD55PohusaBNviGoOJ67HC6Btry4qXLVZc=
```

Dart HMAC SHA256

Dependent upon the Dart crypto package.

```
import 'dart:html';
import 'dart:convert';
import 'package:crypto/crypto.dart';

void main() {

  String secret = 'secret';
  String message = 'Message';

  List<int> secretBytes = UTF8.encode('secret');
  List<int> messageBytes = UTF8.encode('Message');

  var hmac = new HMAC(new SHA256(), secretBytes);
  hmac.add(messageBytes);
  var digest = hmac.close();

  var hash = CryptoUtils.bytesToBase64(digest);

  // output to html page
  querySelector('#hash').text = hash;
  // hash => qnR8UCqJggD55PohusaBNviGoOJ67HC6Btry4qXLVZc=
}
```

Prepared by:
iPay Business Unit,
3rd Floor,
No 438, Havelock Road,
Colombo 05
Sri Lanka

Contact Details
Telephone : (+94) 115 714 444
Email : customersupport@ipayglobal.ai
Website : www.ipay.lk

********** End of Document **********